

1. **Write a program to sort the elements of an array using Randomized Quick Sort (the program should report the number of comparisons).**

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int comparisons = 0;

int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        comparisons++;
        if (arr[j] <= pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

int randomPartition(vector<int>& arr, int low, int high) {
    int random = low + rand() % (high - low + 1);
    swap(arr[random], arr[high]);
    return partition(arr, low, high);
}

void randomizedQuickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = randomPartition(arr, low, high);
        randomizedQuickSort(arr, low, pi - 1);
        randomizedQuickSort(arr, pi + 1, high);
    }
}

int main() {
    vector<int> arr = {5, 2, 8, 1, 9, 3, 7, 6, 4};
    comparisons = 0;

    randomizedQuickSort(arr, 0, arr.size() - 1);

    cout << "Sorted array: ";
    for (int x : arr) cout << x << " ";
    cout << "\nNumber of comparisons: " << comparisons << endl;

    return 0;
}
```

2. Write a program to find the ith smallest element of an array using Randomized Select.

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int partition(vector<int>& arr, int left, int right) {
    int pivot = arr[right];
    int i = left - 1;
    for (int j = left; j < right; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[right]);
    return i + 1;
}

int randomizedPartition(vector<int>& arr, int left, int right) {
    int pivotIndex = left + rand() % (right - left + 1);
    swap(arr[pivotIndex], arr[right]);
    return partition(arr, left, right);
}

int randomizedSelect(vector<int>& arr, int left, int right, int i) {
    if (left == right) return arr[left];

    int pivotIndex = randomizedPartition(arr, left, right);
    int k = pivotIndex - left + 1;

    if (i == k) return arr[pivotIndex];
    else if (i < k) return randomizedSelect(arr, left, pivotIndex - 1, i);
    else return randomizedSelect(arr, pivotIndex + 1, right, i - k);
}

int main() {
    vector<int> arr = {7, 10, 4, 3, 20, 15};
    int i = 3;

    int result = randomizedSelect(arr, 0, arr.size() - 1, i);

    cout << i << "th smallest element: " << result << endl;

    return 0;
}
```

3. Write a program to determine the minimum spanning tree of a graph using Kruskal's algorithm.

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Edge {
    int u, v, weight;
    bool operator<(const Edge& other) const {
        return weight < other.weight;
    }
};

class DisjointSet {
    vector<int> parent, rank;
public:
    DisjointSet(int n) {
        parent.resize(n);
        rank.resize(n, 0);
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]);
        return parent[x];
    }

    void unite(int x, int y) {
        int rootX = find(x);
        int rootY = find(y);
        if (rootX != rootY) {
            if (rank[rootX] < rank[rootY]) parent[rootX] = rootY;
            else if (rank[rootX] > rank[rootY]) parent[rootY] = rootX;
            else {
                parent[rootY] = rootX;
                rank[rootX]++;
            }
        }
    }
};

void kruskalMST(int n, vector<Edge>& edges) {
    sort(edges.begin(), edges.end());
    DisjointSet ds(n);
    vector<Edge> mst;
    int totalWeight = 0;

    for (auto& edge : edges) {
        if (ds.find(edge.u) != ds.find(edge.v)) {
            mst.push_back(edge);
            totalWeight += edge.weight;
        }
    }
}
```

```

        ds.unite(edge.u, edge.v);
    }
}

cout << "Edges in MST:\n";
for (auto& e : mst) {
    cout << e.u << " - " << e.v << " : " << e.weight << endl;
}
cout << "Total weight: " << totalWeight << endl;
}

int main() {
    int n = 4;
    vector<Edge> edges = {
        {0, 1, 10}, {0, 2, 6}, {0, 3, 5},
        {1, 3, 15}, {2, 3, 4}
    };

    kruskalMST(n, edges);

    return 0;
}

```

4. Write a program to implement the Bellman-Ford algorithm to find the shortest paths from a given source node to all other nodes in a graph.

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;

struct Edge {
    int u, v, weight;
};

void bellmanFord(int n, vector<Edge>& edges, int src) {
    vector<int> dist(n, INT_MAX);
    dist[src] = 0;

    for (int i = 0; i < n - 1; i++) {
        for (auto& e : edges) {
            if (dist[e.u] != INT_MAX && dist[e.u] + e.weight < dist[e.v]) {
                dist[e.v] = dist[e.u] + e.weight;
            }
        }
    }

    for (auto& e : edges) {
        if (dist[e.u] != INT_MAX && dist[e.u] + e.weight < dist[e.v]) {
            cout << "Negative weight cycle detected\n";
            return;
        }
    }

    cout << "Vertex\tDistance from Source\n";
    for (int i = 0; i < n; i++) {
        cout << i << "\t";
        if (dist[i] == INT_MAX) cout << "INF\n";
        else cout << dist[i] << "\n";
    }
}

int main() {
    int n = 5;
    vector<Edge> edges = {
        {0, 1, -1}, {0, 2, 4}, {1, 2, 3},
        {1, 3, 2}, {1, 4, 2}, {3, 2, 5},
        {3, 1, 1}, {4, 3, -3}
    };
    int src = 0;

    bellmanFord(n, edges, src);

    return 0;
}
```

5. Write a program to implement a B-Tree.

```
#include <iostream>
using namespace std;

class BTreeNode {
    int *keys;
    int t;
    BTreeNode **children;
    int n;
    bool leaf;
public:
    BTreeNode(int _t, bool _leaf);
    void traverse();
    BTreeNode *search(int k);
    void insertNonFull(int k);
    void splitChild(int i, BTreeNode *y);
    friend class BTree;
};

class BTree {
    BTreeNode *root;
    int t;
public:
    BTree(int _t) { root = NULL; t = _t; }
    void traverse() { if (root != NULL) root->traverse(); }
    BTreeNode* search(int k) { return (root == NULL) ? NULL : root->search(k); }
    void insert(int k);
};

BTreeNode::BTreeNode(int _t, bool _leaf) {
    t = _t;
    leaf = _leaf;
    keys = new int[2*t-1];
    children = new BTreeNode *[2*t];
    n = 0;
}

void BTreeNode::traverse() {
    int i;
    for (i = 0; i < n; i++) {
        if (leaf == false)
            children[i]->traverse();
        cout << " " << keys[i];
    }
    if (leaf == false)
        children[i]->traverse();
}

BTreeNode *BTreeNode::search(int k) {
    int i = 0;
    while (i < n && k > keys[i])
        i++;
}
```

```

    if (i < n && keys[i] == k)
        return this;
    if (leaf == true)
        return NULL;
    return children[i]->search(k);
}

void BTree::insert(int k) {
    if (root == NULL) {
        root = new BTreeNode(t, true);
        root->keys[0] = k;
        root->n = 1;
    } else {
        if (root->n == 2*t-1) {
            BTreeNode *s = new BTreeNode(t, false);
            s->children[0] = root;
            s->splitChild(0, root);
            int i = 0;
            if (s->keys[0] < k)
                i++;
            s->children[i]->insertNonFull(k);
            root = s;
        } else
            root->insertNonFull(k);
    }
}

void BTreeNode::insertNonFull(int k) {
    int i = n-1;
    if (leaf == true) {
        while (i >= 0 && keys[i] > k) {
            keys[i+1] = keys[i];
            i--;
        }
        keys[i+1] = k;
        n = n+1;
    } else {
        while (i >= 0 && keys[i] > k)
            i--;
        if (children[i+1]->n == 2*t-1) {
            splitChild(i+1, children[i+1]);
            if (keys[i+1] < k)
                i++;
        }
        children[i+1]->insertNonFull(k);
    }
}

void BTreeNode::splitChild(int i, BTreeNode *y) {
    BTreeNode *z = new BTreeNode(y->t, y->leaf);
    z->n = t - 1;
    for (int j = 0; j < t-1; j++)
        z->keys[j] = y->keys[j+t];
    if (y->leaf == false) {

```

```

        for (int j = 0; j < t; j++)
            z->children[j] = y->children[j+t];
    }
    y->n = t - 1;
    for (int j = n; j >= i+1; j--)
        children[j+1] = children[j];
    children[i+1] = z;
    for (int j = n-1; j >= i; j--)
        keys[j+1] = keys[j];
    keys[i] = y->keys[t-1];
    n = n + 1;
}

int main() {
    BTree t(3);
    t.insert(10);
    t.insert(20);

    t.insert(5);
    t.insert(6);
    t.insert(12);
    t.insert(30);
    t.insert(7);
    t.insert(17);

    cout << "Traversal of the B-Tree:";
    t.traverse();
    cout << endl;

    int k = 6;
    (t.search(k) != NULL) ? cout << "\nKey " << k << " found" : cout << "\nKey " << k << "
not found";

    k = 15;
    (t.search(k) != NULL) ? cout << "\nKey " << k << " found" : cout << "\nKey " << k << "
not found";
    cout << endl;

    return 0;
}

```


6. **Write a program to search a pattern in a given text using the KMP algorithm.**

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

void computeLPSArray(string pattern, vector<int>& lps) {
    int len = 0;
    int m = pattern.length();
    lps[0] = 0;
    int i = 1;

    while (i < m) {
        if (pattern[i] == pattern[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }
}

void KMPSearch(string text, string pattern) {
    int n = text.length();
    int m = pattern.length();

    vector<int> lps(m);
    computeLPSArray(pattern, lps);

    int i = 0;
    int j = 0;
    int count = 0;

    while (i < n) {
        if (pattern[j] == text[i]) {
            j++;
            i++;
        }

        if (j == m) {
            cout << "Pattern found at index " << i - j << endl;
            count++;
            j = lps[j - 1];
        } else if (i < n && pattern[j] != text[i]) {
            if (j != 0)
                j = lps[j - 1];
        }
    }
}
```

```
        else
            i = i + 1;
    }
}

if (count == 0)
    cout << "Pattern not found" << endl;
else
    cout << "Total occurrences: " << count << endl;
}

int main() {
    string text = "ABABDABACDABABCABAB";
    string pattern = "ABABCABAB";

    cout << "Text: " << text << endl;
    cout << "Pattern: " << pattern << endl;

    KMPSearch(text, pattern);

    return 0;
}
```

7. **Write a program to implement a Suffix tree (bruteForceSuffixTrie method).**

```
#include <iostream>
#include <string>
#include <map>
using namespace std;
struct TrieNode {
    map<char, TrieNode*> children;
    bool isEndOfSuffix;

    TrieNode() {
        isEndOfSuffix = false;
    }
};
class SuffixTrie {
private:
    TrieNode* root;
    string text;

public:
    SuffixTrie(string T) {
        text = T;
        root = new TrieNode();
        bruteForceSuffixTrie(T);
    }

    void bruteForceSuffixTrie(string T) {
        int n = T.length();

        for (int i = n - 1; i >= 0; i--) {
            TrieNode* node = root;
            int j = i;

            while (j < n && node->children.find(T[j]) != node->children.end()) {
                node = node->children[T[j]];
                j++;
            }

            for (int k = j; k < n; k++) {
                TrieNode* newNode = new TrieNode();
                node->children[T[k]] = newNode;
                node = newNode;
            }

            node->isEndOfSuffix = true;
        }
    }

    bool search(string pattern) {
        TrieNode* node = root;

        for (char c : pattern) {
            if (node->children.find(c) == node->children.end()) {
                return false;
            }
        }
    }
};
```

```

        }
        node = node->children[c];
    }

    return true;
}

void displayTrie(TrieNode* node, string prefix, int depth = 0) {
    if (node == nullptr) return;

    if (node->isEndOfSuffix) {
        cout << string(depth * 2, ' ') << "[" << prefix << "]" (End of suffix)" << endl;
    }

    for (auto& pair : node->children) {
        cout << string(depth * 2, ' ') << prefix << " -> " << pair.first << endl;
        displayTrie(pair.second, prefix + pair.first, depth + 1);
    }
}

void display() {
    cout << "\nSuffix Trie Structure:\n";
    displayTrie(root, "", 0);
}

};

int main() {
    string text = "banana$";

    cout << "Building Suffix Trie for text: " << text << endl;
    SuffixTrie trie(text);

    trie.display();

    cout << "\n--- Pattern Search Tests ---\n";

    string patterns[] = {"ana", "nan", "ban", "apple", "na", "$"};

    for (string pattern : patterns) {
        if (trie.search(pattern)) {
            cout << "Pattern \"" << pattern << "\" is a substring" << endl;
        } else {
            cout << "Pattern \"" << pattern << "\" is NOT a substring" << endl;
        }
    }

    return 0;
}

```